

به نام خدا

پروژه درس هوش مصنوعی و سیستم های خبره

الگوریتم Q-Learning

ساختار کلی محیط *Gym*:

هر محیط در *Gym* شامل اجزای زیر است:

- **Observation Space**: فضای حالت‌ها
- **Action Space**: فضای اعمال
- **Reward**: پاداش دریافتی از محیط
- **Termination/Truncated**: پایان اپیزود

عامل در هر گام، یک عمل را انتخاب کرده و محیط با ارائه حالت جدید و پاداش پاسخ می‌دهد.

توابع اصلی محیط *Gym*:

1. تابع `reset`:

تابع `reset()` برای مقداردهی اولیه محیط استفاده می‌شود و معمولاً در ابتدای هر اپیزود فراخوانی می‌گردد.

خروجی

این تابع دو خروجی دارد:

- **observation**: حالت اولیه محیط
- **info**: اطلاعات کمکی

همچنین امکان تعیین **seed** برای بازتولیدپذیری نتایج وجود دارد.

```
obs, info = env.reset()
```

```
obs, info = env.reset(seed=42)
```

2. تابع `step`:

تابع `step` مهم‌ترین تابع محیط است که تعامل اصلی عامل و محیط را شکل می‌دهد.

ورودی

- **action**: عملی که عامل انتخاب می‌کند

خروجی

- **observation**: حالت جدید (دید خود عامل نسبت به محیط)
- **reward**: پاداش دریافتی

- **terminated**: پایان طبیعی اپیزود
- **truncated**: پایان به دلیل محدودیت
- **info**: اطلاعات اضافی

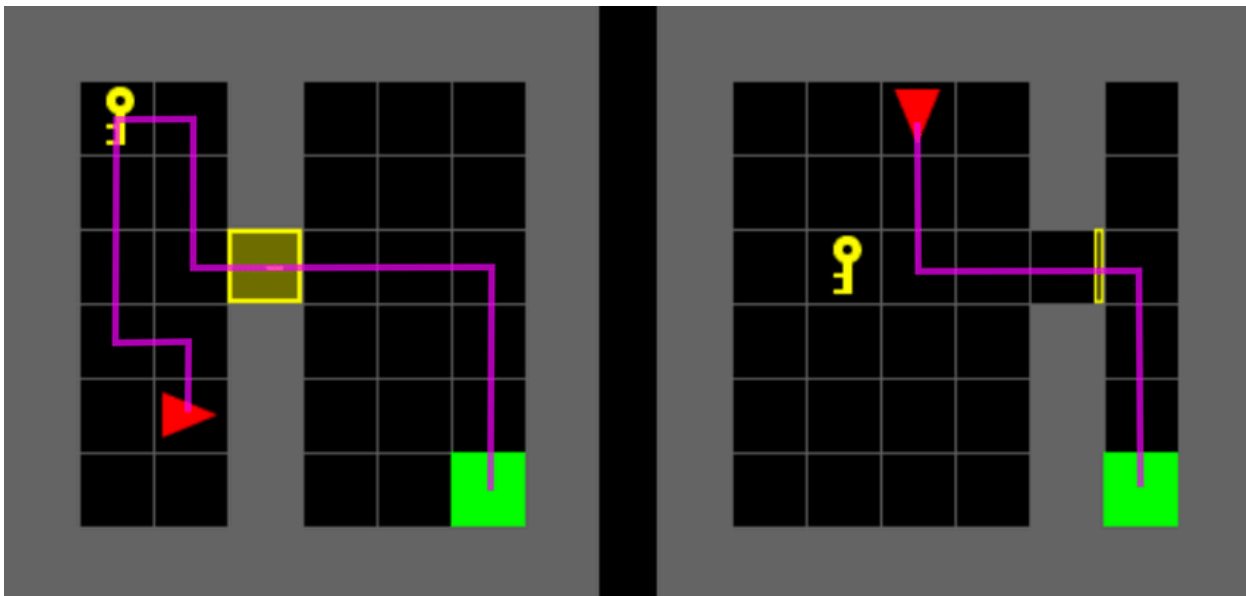
اپیزود زمانی پایان می‌یابد که یکی از متغیرهای terminated یا truncated برابر با True شود.

`obs, reward, terminated, truncated, info = env.step(action)`

محیط *Minigrid-DoorKey*:

در این محیط، عامل باید ابتدا کلیدی را بردارد تا بتواند در را باز کند و سپس به خانه سبز هدف برسد. اگر در باز باشد و کلیدی در محیط قرار داشته باشد عامل نباید به سمت کلید حرکت کند.

وظیفه شما پیاده سازی یک الگوریتم Q-Learning برای آموزش عامل می باشد.



توابع تعریف شده مهم:

1. **preprocess_obs**: تبدیل observation خام محیط به یک state قابل استفاده برای Q-learning.
2. **reward_shaping**: بهبود سیگنال پاداش برای یادگیری سریع‌تر عامل.
3. **select_action**: انتخاب عمل با استفاده از سیاست ϵ -greedy.
4. **update**: به‌روزرسانی مقدار Q با استفاده از قانون Q-learning.
5. **train**: آموزش عامل در چندین اپیزود.
6. **test**: ارزیابی عملکرد عامل آموزش‌دیده.

بخش عملی:

در این بخش وظیفه شما پیاده سازی یک الگوریتم Q-Learning می باشد. شبه کد این الگوریتم به شرح زیر می باشد:

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

```
Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$ 
Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$ 
Loop for each episode:
  Initialize  $S$ 
  Loop for each step of episode:
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)
    Take action  $A$ , observe  $R, S'$ 
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$ 
     $S \leftarrow S'$ 
  until  $S$  is terminal
```

در پیاده سازی به نکات زیر دقت کنید:

1. مقدار reward بازگشتی از تابع step نیازمند تغییر است. تابع reward_shaping به این منظور پیاده سازی شده است.
2. مقدار observation بازگشتی از تابع step نیازمند تغییر است. تابع preprocess_obs به این منظور پیاده سازی شده است.
3. پس از پایان هر اپیزود کاهش ε الزامی است:

$$\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon * \text{decay})$$

بخش نظری:

سوال ۱.

در هنگام آموزش مقادیر زیر چه تغییری می کنند؟ چرا؟

- Avg Reward: میانگین کل پاداش دریافتی در ۱۰۰ اپیزود آخر
- Success: درصد اپیزودهای موفق در ۱۰۰ اپیزود اخیر (اگر پاداش کل اپیزود از 0.5 بیشتر باشد اپیزود موفق در نظر گرفته می شود).
- Avg Steps: میانگین تعداد گام هایی که عامل برای پایان اپیزود استفاده کرده است (در ۱۰۰ اپیزود آخر)

سوال ۲.

۴ نمودار نمایش داده شده پس از آموزش عامل را تحلیل کنید.

سوال ۳.

(برای پاسخ به سوالات این بخش تابع `select_action` کمک کننده خواهد بود.)

مقدار `epsilon` چه تغییری در حین آموزش کرده است؟

مقدار آن چه تاثیری در فرایند آموزش دارد؟

دلیل استفاده از `epsilon_decay_rate` در فرایند آموزش چیست؟

چرا در تابع `test` (هنگام ارزیابی عامل پس از آموزش) مقدار `epsilon` صفر لحاظ شده است؟

سوال ۴.

اگر در توابع `train` و `test` به جای

`env.reset()`

قرار دهیم:

`env.reset(seed=42)`

باعث تکرار پذیری محیط در هر اپیزود می شود.

این کار چه تغییری در آموزش عامل ایجاد می کند؟

۴ نمودار خروجی پس از آموزش را تحلیل کنید.

صحیح/ غلط بودن جملات زیر را مشخص کرده و برای آن دلیل بیاورید.

1. مقدار `seed=42` تضمین می کند که تمام `randomness` در طول کل اپیزود کاملاً `deterministic` باشد.

2. در سیستم های RL همواره باید از مقدار `seed=42` استفاده کرد.